Web Development Tooling & Workflows

Complete Student Study Guide & Technical Manual

Full Course Reference

1. PARADIGM SHIFT: CDN PROTOTYPING VS. LOCAL BUILD ENVIRONMENTS

Web development begins with fast CDN prototyping but scales to local build environments for real-world production applications.

Workflow Dimension	CDN Link (Content Delivery Network)	Local NPM & Build Pipeline (Vite)
Mechanism	Loads pre-compiled files via HTTP `<link>` or `<script>` tags.	Downloads package sources via NPM into `node_modules` and compiles locally.
Primary Use Case	Quick prototypes, classroom testing, single HTML file demos.	Production web applications, team projects, optimized deployments.
Reliability & Risk	Subject to third-party network outages or breaking external updates.	Strictly version-locked locally via `package.json` and lockfiles.
Performance & Purging	Loads full library files containing unused styles and scripts.	Tree-shakes and purges unused CSS/JS to generate minimal production bundles.

2. STYLING SOLUTIONS: BOOTSTRAP VS. TAILWIND CSS

Bootstrap (Component-Driven)

- **Approach:** Pre-designed UI components (e.g., `.btn`, `.card`, `.navbar`).
- **Pros:** Fast setup out of the box; built-in JavaScript components.
- **Cons:** Sites look visually identical; harder to customize deeply.
- **Local Setup:** `npm install bootstrap`

Tailwind CSS (Utility-First)

- **Approach:** Low-level utility classes (e.g., `flex`, `p-4`, `bg-purple-200`).
- **Pros:** Infinite UI customization directly inside HTML; zero bloated CSS.
- **Cons:** Verbose HTML class attributes.
- **Local Setup:** `npm install tailwindcss @tailwindcss/vite`

3. DEEP DIVE INTO NPM (NODE PACKAGE MANAGER)

NPM manages external dependencies and automates development tasks inside JavaScript project trees.

Core Files & Storage

- **`package.json`**: The project recipe file tracking metadata, run scripts, and dependency version lists.
- **`node_modules/`**: Local directory housing thousands of library files. Always added to `.gitignore`.

Command Line Reference

- `npm init -y` — Scaffolds default `package.json`.
- `npm i <pkg>` — Installs dependency locally.
- `npm i -D <pkg>` — Installs dev-only tool (e.g. `http-server`).
- `npx <cmd>` — Executes binary package on the fly.
- `npm run <script>` — Triggers scripts declared in `package.json`.

⚠️ System Permissions & Global Install Mechanics (EACCES Error)

Running `npm i -g <package>` attempts to write files directly into system-level directories like `/usr/local/lib/node_modules/`. On macOS/Linux, standard users lack administrative write access, triggering `npm error code EACCES (errno -13)`.

- **Modern Preferred Fix:** Skip global installation entirely and use project-level installs or `npx` (e.g., `npx http-server`).
- **Administrator Override:** Prepend `sudo` (e.g., `sudo npm i -g http-server`) and enter system user password.

4. STEP-BY-STEP IMPLEMENTATION BLUEPRINT: VITE + TAILWIND CSS V4

Execute this sequence in terminal and editor to construct the `tw-app` local dev server:

Step A: Terminal Setup & Dependency Installation

```
# 1. Create a fresh Vite project using Vanilla JS template
npm create vite tw-app -- --template vanilla

# 2. Enter project root & open editor
cd tw-app
code .

# 3. Install core dependencies & Tailwind v4 Vite plugin
npm install
npm install tailwindcss @tailwindcss/vite
```

Step B: Configure `vite.config.js`

```
import { defineConfig } from "vite";
import tailwindcss from "@tailwindcss/vite";

export default defineConfig({
  plugins: [tailwindcss()],
  server: {
    open: true // Auto-opens browser
  }
});
```

Step C: Setup CSS Entry Point

In `src/style.css`, clear boilerplate and add:

```
@import "tailwindcss";
```

Start development server:

```
npm run dev
# Stop server: Ctrl + C
```

Step D: Production HTML Structure with Utilities (`index.html`)

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>tw-app</title>
    <link rel="stylesheet" href="src/style.css">
  </head>
  <body>
    <div id="alert" class="container max-w-md mx-auto mt-10">
      <div class="p-4 bg-purple-200 dark:bg-purple-800 text-purple-800 dark:text-purple-200 rounded-lg
border border-purple-300 dark:border-purple-600">
        Lorem ipsum dolor sit amet.
      </div>
      <ul class="mt-5 bg-slate-200 dark:bg-slate-700 p-2 rounded-lg">
        <li class="p-3 bg-white dark:bg-slate-800 dark:text-white mb-2 rounded">Item One</li>
        <li class="p-3 bg-white dark:bg-slate-800 dark:text-white mb-2 rounded">Item Two</li>
        <li class="p-3 bg-white dark:bg-slate-800 dark:text-white rounded">Item Three</li>
      </ul>
    </div>
  </body>
</html>
```